PHASE 1 — Operational Pricing Engine

Deep Product & Technical Architecture Specification (Part 1)

---

1. Phase Objective

Purpose of Phase 1

Phase 1 focuses ONLY on building the foundational pricing intelligence brain of the platform.

This phase is NOT about:

- proposal uploads
- AI audits
- quotation archives
- benchmarking intelligence
- historical analytics
- project tracking

This phase ONLY builds:

the operational pricing engine.

The operational pricing engine becomes:

- the financial foundation
- the cost intelligence layer
- the profitability engine
- the safe billing computation system

Every future module depends on this layer.

If this phase is architected correctly:

- quotation quality improves
- underquoting reduces
- operational visibility improves
- profitability becomes predictable

---

2. Core Product Philosophy

The platform must NOT behave like:

- a calculator
- a spreadsheet

- a finance dashboard
- a simple quote tool

The platform should behave like:

a live operational profitability simulator.

Every financial component should influence:

- pricing
- margin
- hourly burn
- quote safety
- profitability sustainability

The system should continuously answer:

“What is the minimum financially safe pricing structure for this organization?”

---

3. Product Scope (Phase 1 Only)

Phase 1 contains ONLY:

Included Modules

1. Workforce Economics Engine

2. Infrastructure Cost Engine

3. AI & SaaS Allocation Engine

4. Operational Overhead Engine

5. Productivity Modeling Engine

6. Margin Intelligence Engine

7. Safe Billing Rate Engine

8. Operational Health Engine

9. Financial Configuration Workspace

10. Pricing Simulation Logic

---

4. Core Architecture Overview

SYSTEM FLOW

Organizational Inputs

↓

Operational Cost Intelligence

↓

Productivity Modeling

↓

Effective Hourly Burn

↓

Margin Logic

↓

Safe Billing Computation

↓

Pricing Safety Output

---

5. Operational Philosophy

The entire engine is based on one core principle:

Employees are not fully billable.

Most agencies incorrectly calculate:

Salary / Working Hours

This is financially wrong.

Real delivery organizations lose hours due to:

- meetings
- operations
- context switching
- leave
- sales support
- grooming
- internal discussions
- learning
- documentation
- inefficiencies

Therefore:

Productive hours must be modeled.

This becomes the core intelligence layer.

---

6. Workforce Economics Engine

Purpose

This module computes:

- true employee delivery cost
- effective hourly burn
- utilization-adjusted cost
- productivity-adjusted cost

This is the MOST important module in the platform.

---

7. Workforce Data Structure

ENTITY: EmployeeRole

Fields

Field	Type	Description
id	UUID	Unique identifier
role_name	String	Senior Developer
department	String	Engineering
annual_salary	Number	Total annual salary
monthly_salary	Number	Auto calculated
productive_hours_month	Number	Actual productive hrs
total_working_hours_month	Number	Standard hours
utilization_percent	Number	Delivery utilization
allocation_factor	Number	Shared cost allocation
overhead_multiplier	Number	Optional burden factor
effective_hourly_cost	Number	System calculated
active_status	Boolean	Enabled/disabled
created_at	Timestamp	Record timestamp

---

8. Workforce Configuration UX

Users should be able to:

Actions

- Add role
- Edit role
- Delete role
- Clone role
- Disable role
- Save reusable role templates

Example Roles

- Senior Developer
- Junior Developer
- UI/UX Designer
- QA Engineer
- DevOps Engineer
- Product Manager
- AI Engineer
- Data Engineer
- Technical Architect

---

9. Productive Hours Logic

IMPORTANT

This becomes one of the biggest differentiators.

The system should NOT assume:

8 hrs/day × 22 days

Instead:

Productive Delivery Hours

Must account for:

- meetings
- operational overhead

- breaks
- coordination
- documentation
- internal support
- learning
- management time

---

10. Productive Hours Formula

Core Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Productive\ Hours = Total\ Working\ Hours - NonBillable\ Hours"}}

Where:

NonBillable Hours =

Meetings + Operations + Leave + Internal Support + Learning

---

11. Effective Hourly Cost Formula

This is the CORE ENGINE FORMULA.

Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Effective\ Hourly\ Cost = \frac{Monthly\ Salary + Cost\ Allocation}{Productive\ Hours}"}}

Example

Employee

- Salary = ₹120,000/month
- Productive Hours = 120 hrs/month
- Operational Allocation = ₹15,000

Output

Effective Hourly Cost:

(120000 + 15000) / 120

= ₹1125/hr

This becomes:

the minimum internal burn rate.

---

12. Utilization Modeling

The system must model:

actual delivery utilization.

Utilization Definition

Utilization indicates:

“How much of an employee’s available productive capacity contributes to billable delivery?”

Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Utilization\ Percentage = \frac{Billable\ Hours}{Available\ Productive\ Hours} \times 100"}}

Impact Logic

Lower utilization increases:

- effective burn
- delivery pressure
- quote risk

The engine must dynamically adjust:

- hourly cost
- safe billing rate
- operational margin

based on utilization changes.

---

13. Infrastructure Cost Engine

Purpose

This module computes:

- cloud costs
- hosting burden
- infra allocation
- platform delivery cost

ENTITY: InfrastructureService

Fields

Field	Type
id	UUID
service_name	String
category	String
monthly_cost	Number
allocation_type	Enum
allocation_percent	Number
team_mapping	String
project_dependency	Boolean
active_status	Boolean

Infrastructure Categories

Examples:

- AWS
- Azure
- Vercel
- Cloudflare
- MongoDB Atlas
- Supabase
- Firebase
- Render
- GPU Hosting
- CDN
- Monitoring

Infrastructure Allocation Logic

Infrastructure can be allocated:

Allocation Types

1. Organization-Wide

Spread across all projects.

2. Department-Wise

Engineering-only allocation.

3. Project-Specific

Direct project dependency.

4. Usage-Based

Based on estimated infra intensity.

Infrastructure Cost Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Allocated\ Infrastructure\ Cost = Monthly\ Cost \times Allocation\ Percentage"}}

---

14. AI & SaaS Cost Engine

Purpose

Compute:

- AI operational burn
- tooling cost allocation
- software delivery burden

ENTITY: SaaSTool

Fields

Field	Type
id	UUID
tool_name	String
category	String
monthly_cost	Number
seats	Number
allocation_percent	Number

Field	Type
organization_wide	Boolean
ai_dependency	Boolean
active_status	Boolean

Example Tools

AI

- ChatGPT
- Claude
- Gemini
- Cursor

Productivity

- Notion
- Slack
- Jira
- ClickUp
- Figma

Dev

- GitHub
- Postman
- Docker

SaaS Allocation Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Allocated\ SaaS\ Cost = Total\ Tool\ Cost \times Usage\ Allocation"}}

---

15. Operational Overhead Engine

Purpose

Capture hidden business burden.

Most agencies ignore:

- admin costs

- accounting
- office costs
- HR burden
- compliance
- tax-related costs

This creates:

hidden profitability erosion.

ENTITY: OverheadCategory

Fields

Field	Type
id	UUID
category_name	String
monthly_cost	Number
allocation_logic	Enum
recurring	Boolean
operational_criticality	Enum

Example Overheads

- Office rent
- HR
- Recruitment
- Internet
- Utilities
- Accounting
- Compliance
- Legal
- Insurance
- Management overhead

Overhead Allocation Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Operational\ Overhead\ Allocation = Total\ Overheads \div Active\ Delivery\ Capacity"}}

---

16. Productivity Modeling Engine

Purpose

This engine models:

- delivery efficiency
- non-billable drain
- operational leakage
- execution friction

This becomes:

advanced profitability intelligence.

Productivity Inputs

Required Inputs

- meeting percentage
- management overhead
- operational coordination
- leave factor
- context switching
- support dependency
- internal operations

Productivity Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Delivery\ Efficiency = \frac{Productive\ Hours}{Total\ Available\ Hours}"}}

Output Impact

Low efficiency increases:

- operational burn
- quote risk

- safe billing requirement
- pricing pressure

---

17. Real-Time Dependency System

IMPORTANT.

The platform must behave:

reactively.

Meaning:
When any value changes:

- salary
- utilization
- infra
- SaaS
- overheads
- productive hours

The entire pricing system recalculates instantly.

Dynamic Dependency Flow

Salary Change

↓

Hourly Burn Updates

↓

Safe Billing Rate Changes

↓

Margin Threshold Changes

↓

Quote Safety Updates

This creates:

simulation behavior.

---

18. Operational Health Engine

Purpose

Generate:

- operational stability indicators
- burn sustainability
- pricing pressure analysis
- workforce health

Operational Health Factors

Inputs

- utilization
- operational burn
- productivity efficiency
- overhead ratio
- dependency risk
- infrastructure intensity

Outputs

Operational Status

- Healthy
- Moderate Risk
- High Burn Risk
- Unsustainable

---

19. Configuration Persistence

IMPORTANT.

All operational data must be:

- manually editable
- reusable
- saveable
- versionable

Save Logic

Users should be able to:

Save

- workforce presets
- infra presets
- SaaS stacks
- regional pricing profiles
- operational templates

Example Use Cases

Example

Moonhive Enterprise Setup

Contains:

- workforce structure
- AI stack
- infra costs
- productivity assumptions
- default margins

Reusable for all future estimates.

---

20. Core UX Philosophy

The engine should NOT feel like:

- finance admin software
- ERP configuration
- spreadsheet entry

It should feel like:

configuring a live pricing intelligence system.

The user should continuously feel:

- financially aware
- operationally informed
- strategically guided

- pricing confident

---

21. Margin Intelligence Engine

Purpose

This engine controls:

- profitability safety
- pricing aggressiveness
- minimum acceptable margin
- contingency behavior
- enterprise pricing logic

This becomes:

the strategic pricing layer.

IMPORTANT PRINCIPLE

The platform must distinguish between:

Markup

and

Margin

Most agencies incorrectly use markup thinking it is margin.

This creates:

- false profitability assumptions
- hidden losses
- pricing distortion

The platform must educate and calculate correctly.

Margin Formula

TRUE MARGIN FORMULA

genui {"math_block_widget_always_prefetch_v2":{"content":"Margin\ Percentage = \frac{Revenue - Cost}{Revenue} \times 100"}}

Markup Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Markup\ Percentage = \frac{Revenue - Cost}{Cost} \times 100"}}

Margin Configuration Rules

ENTITY: MarginPolicy

Fields

Field	Type
id	UUID
policy_name	String
minimum_safe_margin	Number
target_margin	Number
enterprise_margin	Number
contingency_default	Number
emergency_buffer	Number
pricing_mode	Enum
active_status	Boolean

Pricing Modes

Conservative

Higher safety.
Higher contingency.

Balanced

Balanced pricing strategy.

Aggressive

Competitive pricing.
Lower margin tolerance.

Margin Sustainability Logic

The platform should compute:

Margin Stability

Factors:

- operational burn
- delivery duration
- payment recovery
- utilization
- infra dependency
- scope uncertainty

Margin Risk Levels

Safe

Margin above threshold.

Moderate

Margin near minimum threshold.

Risky

Margin vulnerable to small delivery variation.

Unsustainable

High probability of loss.

---

22. Contingency Intelligence Engine

Purpose

Most agencies underestimate:

- scope creep
- iteration cycles
- delivery delays
- communication overhead
- rework
- client-side inefficiencies

This engine introduces:

operational safety buffering.

Contingency Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Contingency\ Allocation = Operational\ Cost imes Contingency\ Percentage"}}

Recommended Contingency Logic

Small Projects

5%–10%

Medium Projects

10%–15%

Enterprise AI Projects

15%–25%

Dynamic Contingency Factors

The engine should increase contingency based on:

- unclear scope
- AI dependency
- high stakeholder count
- short delivery timeline
- unknown integrations
- infrastructure complexity
- multi-region deployment

AI Recommendation Example

High integration complexity detected.

Recommended contingency increased from 10% → 16%.

---

23. Safe Billing Rate Engine

Purpose

Compute:

minimum financially safe hourly pricing.

This is one of the MOST important outputs in the platform.

The product exists primarily to compute:

sustainable billing rates.

Core Safe Billing Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Safe\ Billing\ Rate = \frac{True\ Project\ Cost}{Estimated\ Delivery\ Hours}"}}

Enhanced Safe Billing Logic

The final billing rate must account for:

- workforce burn
- infra burden
- SaaS allocation
- operational overhead
- contingency
- target margin
- utilization pressure
- payment risk

Full Pricing Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Recommended\ Quote = \frac{Operational\ Cost + Contingency}{1 - Target\ Margin}"}}

Billing Intelligence Outputs

Outputs

- minimum safe billing rate
- ideal billing rate
- enterprise billing range
- aggressive pricing threshold
- benchmark alignment

Pricing Zones

Unsafe Zone

Below sustainability threshold.

Competitive Zone

Healthy and competitive.

Premium Zone

Enterprise-level positioning.

Overpriced Zone

Risk of reduced conversion.

---

24. Regional Pricing Intelligence Engine

Purpose

Pricing expectations vary by geography.

The platform must model:

regional commercial positioning.

ENTITY: RegionalBenchmark

Fields

Field	Type
id	UUID
region_name	String
currency	String
minimum_rate	Number
average_rate	Number
enterprise_rate	Number
competitiveness_index	Number
updated_at	Timestamp

Example Benchmarks

Region	Avg Rate
US	$145/hr

Region	Avg Rate
UK	£110/hr
UAE	$120/hr
India	₹2500/hr

Benchmark Positioning Logic

The system compares:

Generated Billing Rate

vs

Regional Benchmark Range

Benchmark Output Types

Underpriced

Below sustainable market range.

Competitive

Aligned with regional expectation.

Premium

High-value positioning.

Overpriced

Above commercial expectation.

---

25. Cash Flow & NPV Engine

Purpose

Revenue timing matters.

A ₹10L project paid after 6 months is NOT equal to:
₹10L received upfront.

The platform must model:

time-adjusted profitability.

Milestone Architecture

ENTITY: MilestoneStructure

Fields

Field	Type
id	UUID
project_id	UUID
milestone_name	String
percentage	Number
due_date	Date
payment_delay_days	Number
expected_cash_flow	Number

Example Milestones

- 30% upfront
- 40% mid delivery
- 30% final delivery

NPV Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"NPV = \sum \frac{Cash\ Flow_t}{(1+r)^t}"}}

NPV Variables

Variable	Meaning
r	discount rate
t	payment period
Cash Flow	milestone payment

NPV Intelligence Outputs

Outputs

- present profitability
- delayed recovery impact
- operational funding pressure

- cash sustainability score
- recovery quality

Cash Flow Risk Types

Healthy Recovery

Positive cash recovery.

Delayed Recovery

Operational burden increases.

Funding Pressure

Organization must self-finance delivery.

High Exposure

Severe delayed payment dependency.

---

26. Pricing Risk Intelligence Engine

Purpose

Detect:

- hidden loss risk
- underquote risk
- delivery instability
- operational exposure

Risk Inputs

Inputs

- low margin
- low contingency
- delayed cash flow
- low utilization
- high infra dependency
- high operational overhead
- short timelines
- aggressive pricing

Risk Scoring System

ENTITY: RiskProfile

Fields

Field	Type
risk_score	Number
underquote_probability	Number
profitability_confidence	Number
operational_stability	Number
recovery_health	Number

Risk Output Levels

Healthy

Low operational exposure.

Moderate Risk

Manageable volatility.

High Risk

Significant profitability threat.

Critical

High probability of hidden loss.

---

27. Pricing Simulation Engine

Purpose

The platform must behave:

like a live financial simulator.

NOT:

a static form.

Every pricing variable should recalculate:

- instantly
- visually

- contextually

Simulation Triggers

Trigger Events

- salary changes
- utilization changes
- infra changes
- margin changes
- timeline changes
- project hours changes
- milestone changes
- contingency changes

Simulation Outputs

Dynamic Outputs

- billing rate
- quote recommendation
- margin sustainability
- pricing risk
- profitability
- NPV
- operational burn

Simulation Pipeline

Input Change

↓

Dependency Recalculation

↓

Cost Engine Update

↓

Margin Recompute

↓

Billing Rate Recompute

↓

Risk Recompute

↓

UI Simulation Update

---

28. Frontend State Architecture

IMPORTANT

The frontend must maintain:

centralized financial state.

All pricing modules depend on shared calculations.

Recommended State Structure

Global State Areas

Workforce State

Stores:

- employee roles
- utilization
- hourly burn

Infrastructure State

Stores:

- infra costs
- allocation logic

SaaS State

Stores:

- AI tools
- allocation percentages

Margin State

Stores:

- target margin
- contingency
- pricing mode

Simulation State

Stores:

- live outputs
- risk scores
- quote recommendation
- billing rate

---

29. Backend Architecture

Backend Responsibilities

The backend handles:

- persistence
- recalculation
- dependency orchestration
- versioning
- pricing computation
- benchmark retrieval

Backend Layers

Layer 1 — Configuration Engine

Stores operational settings.

Layer 2 — Calculation Engine

Handles deterministic pricing formulas.

Layer 3 — Simulation Engine

Handles live recalculation.

Layer 4 — Intelligence Layer

Handles AI recommendations.

IMPORTANT PRINCIPLE

Core calculations MUST remain deterministic.

AI must NEVER compute:

- billing rates
- profitability
- operational burn
- margins
- NPV

AI should ONLY provide:

- recommendations
- interpretations
- suggestions
- explanations

---

30. Database Relationship Architecture

Relationship Overview

Organization

↓

Employees

↓

Operational Costs

↓

Pricing Configuration

↓

Simulation Engine

↓

Pricing Outputs

Relationship Model

Organization

Has many:

- employees
- SaaS tools
- infra services
- overhead categories
- pricing policies

Projects

Depend on:

- operational configuration
- pricing policies
- benchmark rules

Simulations

Generated from:

- project inputs
- operational engine
- margin policies

---

31. Recalculation Architecture

IMPORTANT

The platform must avoid:

full-page recomputation.

Use:

dependency-based recalculation.

Example

If:

- AWS cost changes

ONLY recalculate:

- infra allocation
- operational burn
- billing rate
- profitability

DO NOT recompute unrelated systems.

Performance Principle

This becomes critical later for:

- scalability
- responsiveness
- simulation smoothness

---

32. Save & Versioning System

Purpose

Organizations evolve.

Pricing logic changes over time.

The system must support:

versioned financial configurations.

Versioning Use Cases

Examples

- Q1 pricing model
- Enterprise AI pricing model
- Startup pricing model
- UAE pricing strategy

Save Types

Draft

Temporary simulation.

Saved Configuration

Reusable operational setup.

Locked Baseline

Protected organizational standard.

---

33. Antigravity Implementation Guidance

Recommended Build Order

STEP 1

Database schema.

STEP 2

Configuration persistence.

STEP 3

Calculation engine.

STEP 4

Simulation engine.

STEP 5

UI reactivity.

STEP 6

Visualization layer.

STEP 7

AI recommendation layer.

CRITICAL RULE

DO NOT start with:

- AI
- dashboards
- animations
- charts

Build:

financial correctness first.

---

34. Future Scalability Preparation

The architecture should later support:

- multi-org systems
- enterprise pricing models
- AI learning systems
- historical profitability analysis
- proposal auditing
- benchmark intelligence

Therefore:
all core calculations should remain modular.

---

35. Final Phase 1 Objective

At the end of Phase 1:

The system should successfully answer:

Core Question

“What is the minimum financially safe billing structure for this organization based on its actual operational cost reality?”

If the platform solves this correctly:

- underquoting reduces
- pricing confidence improves
- profitability becomes predictable
- quote engineering becomes strategic

That is the TRUE product outcome.

Deep Product & Technical Architecture Specification

---

21. Margin Intelligence Engine

Purpose

This engine controls:

- profitability safety
- pricing aggressiveness

- minimum acceptable margin
- contingency behavior
- enterprise pricing logic

This becomes:

the strategic pricing layer.

IMPORTANT PRINCIPLE

The platform must distinguish between:

Markup

and

Margin

Most agencies incorrectly use markup thinking it is margin.

This creates:

- false profitability assumptions
- hidden losses
- pricing distortion

The platform must educate and calculate correctly.

Margin Formula

TRUE MARGIN FORMULA

genui {"math_block_widget_always_prefetch_v2":{"content":"Margin\ Percentage = \frac{Revenue - Cost}{Revenue} \times 100"}}

Markup Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Markup\ Percentage = \frac{Revenue - Cost}{Cost} \times 100"}}

Margin Configuration Rules

ENTITY: MarginPolicy

Fields

Field	Type

Field	Type
id	UUID
policy_name	String
minimum_safe_margin	Number
target_margin	Number
enterprise_margin	Number
contingency_default	Number
emergency_buffer	Number
pricing_mode	Enum
active_status	Boolean

Pricing Modes

Conservative

Higher safety.
Higher contingency.

Balanced

Balanced pricing strategy.

Aggressive

Competitive pricing.
Lower margin tolerance.

Margin Sustainability Logic

The platform should compute:

Margin Stability

Factors:

- operational burn
- delivery duration
- payment recovery

- utilization
- infra dependency
- scope uncertainty

Margin Risk Levels

Safe

Margin above threshold.

Moderate

Margin near minimum threshold.

Risky

Margin vulnerable to small delivery variation.

Unsustainable

High probability of loss.

---

22. Contingency Intelligence Engine

Purpose

Most agencies underestimate:

- scope creep
- iteration cycles
- delivery delays
- communication overhead
- rework
- client-side inefficiencies

This engine introduces:

operational safety buffering.

Contingency Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Contingency\ Allocation = Operational\ Cost \times Contingency\ Percentage"}}

Recommended Contingency Logic

Small Projects

5%–10%

Medium Projects

10%–15%

Enterprise AI Projects

15%–25%

Dynamic Contingency Factors

The engine should increase contingency based on:

- unclear scope
- AI dependency
- high stakeholder count
- short delivery timeline
- unknown integrations
- infrastructure complexity
- multi-region deployment

AI Recommendation Example

High integration complexity detected.

Recommended contingency increased from 10% → 16%.

---

23. Safe Billing Rate Engine

Purpose

Compute:

minimum financially safe hourly pricing.

This is one of the MOST important outputs in the platform.

The product exists primarily to compute:

sustainable billing rates.

Core Safe Billing Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Safe\ Billing\ Rate = \frac{True\ Project\ Cost}{Estimated\ Delivery\ Hours}"}}

Enhanced Safe Billing Logic

The final billing rate must account for:

- workforce burn
- infra burden
- SaaS allocation
- operational overhead
- contingency
- target margin
- utilization pressure
- payment risk

Full Pricing Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Recommended\ Quote = \frac{Operational\ Cost + Contingency}{1 - Target\ Margin}"}}

Billing Intelligence Outputs

Outputs

- minimum safe billing rate
- ideal billing rate
- enterprise billing range
- aggressive pricing threshold
- benchmark alignment

Pricing Zones

Unsafe Zone

Below sustainability threshold.

Competitive Zone

Healthy and competitive.

Premium Zone

Enterprise-level positioning.

Overpriced Zone

Risk of reduced conversion.

---

24. Regional Pricing Intelligence Engine

Purpose

Pricing expectations vary by geography.

The platform must model:

regional commercial positioning.

ENTITY: RegionalBenchmark

Fields

Field	Type
id	UUID
region_name	String
currency	String
minimum_rate	Number
average_rate	Number
enterprise_rate	Number
competitiveness_index	Number
updated_at	Timestamp

Example Benchmarks

Region	Avg Rate
US	$145/hr
UK	£110/hr
UAE	$120/hr
India	₹2500/hr

Benchmark Positioning Logic

The system compares:

Generated Billing Rate

vs

Regional Benchmark Range

Benchmark Output Types

Underpriced

Below sustainable market range.

Competitive

Aligned with regional expectation.

Premium

High-value positioning.

Overpriced

Above commercial expectation.

---

25. Cash Flow & NPV Engine

Purpose

Revenue timing matters.

A ₹10L project paid after 6 months is NOT equal to:
₹10L received upfront.

The platform must model:

time-adjusted profitability.

Milestone Architecture

ENTITY: MilestoneStructure

Fields

Field	**Type**
id	UUID
project_id	UUID

Field	Type
milestone_name	String
percentage	Number
due_date	Date
payment_delay_days	Number
expected_cash_flow	Number

Example Milestones

- 30% upfront
- 40% mid delivery
- 30% final delivery

NPV Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"NPV = \sum \frac{Cash\ Flow_t}{(1+r)^t}"}}

NPV Variables

Variable	Meaning
r	discount rate
t	payment period
Cash Flow	milestone payment

NPV Intelligence Outputs

Outputs

- present profitability
- delayed recovery impact
- operational funding pressure
- cash sustainability score
- recovery quality

Cash Flow Risk Types

Healthy Recovery

Positive cash recovery.

Delayed Recovery

Operational burden increases.

Funding Pressure

Organization must self-finance delivery.

High Exposure

Severe delayed payment dependency.

---

26. Pricing Risk Intelligence Engine

Purpose

Detect:

- hidden loss risk
- underquote risk
- delivery instability
- operational exposure

Risk Inputs

Inputs

- low margin
- low contingency
- delayed cash flow
- low utilization
- high infra dependency
- high operational overhead
- short timelines
- aggressive pricing

Risk Scoring System

ENTITY: RiskProfile

Fields

Field	Type
risk_score	Number
underquote_probability	Number
profitability_confidence	Number
operational_stability	Number
recovery_health	Number

Risk Output Levels

Healthy

Low operational exposure.

Moderate Risk

Manageable volatility.

High Risk

Significant profitability threat.

Critical

High probability of hidden loss.

---

27. Pricing Simulation Engine

Purpose

The platform must behave:

like a live financial simulator.

NOT:

a static form.

Every pricing variable should recalculate:

- instantly
- visually
- contextually

Simulation Triggers

Trigger Events

- salary changes
- utilization changes
- infra changes
- margin changes
- timeline changes
- project hours changes
- milestone changes
- contingency changes

Simulation Outputs

Dynamic Outputs

- billing rate
- quote recommendation
- margin sustainability
- pricing risk
- profitability
- NPV
- operational burn

Simulation Pipeline

Input Change

↓

Dependency Recalculation

↓

Cost Engine Update

↓

Margin Recompute

↓

Billing Rate Recompute

↓

Risk Recompute

↓

UI Simulation Update

---

28. Frontend State Architecture

IMPORTANT

The frontend must maintain:

centralized financial state.

All pricing modules depend on shared calculations.

Recommended State Structure

Global State Areas

Workforce State

Stores:

- employee roles
- utilization
- hourly burn

Infrastructure State

Stores:

- infra costs
- allocation logic

SaaS State

Stores:

- AI tools
- allocation percentages

Margin State

Stores:

- target margin

- contingency
- pricing mode

Simulation State

Stores:

- live outputs
- risk scores
- quote recommendation
- billing rate

---

29. Backend Architecture

Backend Responsibilities

The backend handles:

- persistence
- recalculation
- dependency orchestration
- versioning
- pricing computation
- benchmark retrieval

Backend Layers

Layer 1 — Configuration Engine

Stores operational settings.

Layer 2 — Calculation Engine

Handles deterministic pricing formulas.

Layer 3 — Simulation Engine

Handles live recalculation.

Layer 4 — Intelligence Layer

Handles AI recommendations.

IMPORTANT PRINCIPLE

Core calculations MUST remain deterministic.

AI must NEVER compute:

- billing rates
- profitability
- operational burn
- margins
- NPV

AI should ONLY provide:

- recommendations
- interpretations
- suggestions
- explanations

---

30. Database Relationship Architecture

Relationship Overview

Organization

↓

Employees

↓

Operational Costs

↓

Pricing Configuration

↓

Simulation Engine

↓

Pricing Outputs

Relationship Model

Organization

Has many:

- employees
- SaaS tools
- infra services
- overhead categories
- pricing policies

Projects

Depend on:

- operational configuration
- pricing policies
- benchmark rules

Simulations

Generated from:

- project inputs
- operational engine
- margin policies

---

31. Recalculation Architecture

IMPORTANT

The platform must avoid:

full-page recomputation.

Use:

dependency-based recalculation.

Example

If:

- AWS cost changes

ONLY recalculate:

- infra allocation
- operational burn
- billing rate
- profitability

DO NOT recompute unrelated systems.

Performance Principle

This becomes critical later for:

- scalability
- responsiveness
- simulation smoothness

---

32. Save & Versioning System

Purpose

Organizations evolve.

Pricing logic changes over time.

The system must support:

versioned financial configurations.

Versioning Use Cases

Examples

- Q1 pricing model
- Enterprise AI pricing model
- Startup pricing model
- UAE pricing strategy

Save Types

Draft

Temporary simulation.

Saved Configuration

Reusable operational setup.

Locked Baseline

Protected organizational standard.

---

33. Antigravity Implementation Guidance

Recommended Build Order

STEP 1

Database schema.

STEP 2

Configuration persistence.

STEP 3

Calculation engine.

STEP 4

Simulation engine.

STEP 5

UI reactivity.

STEP 6

Visualization layer.

STEP 7

AI recommendation layer.

CRITICAL RULE

DO NOT start with:

- AI
- dashboards
- animations
- charts

Build:

financial correctness first.

---

34. Future Scalability Preparation

The architecture should later support:

- multi-org systems
- enterprise pricing models
- AI learning systems
- historical profitability analysis
- proposal auditing
- benchmark intelligence

Therefore:
all core calculations should remain modular.

---

35. Final Phase 1 Objective

At the end of Phase 1:

The system should successfully answer:

Core Question

“What is the minimum financially safe billing structure for this organization based on its actual operational cost reality?”

If the platform solves this correctly:

- underquoting reduces
- pricing confidence improves
- profitability becomes predictable
- quote engineering becomes strategic

That is the TRUE product outcome.

---

36. Smart Estimate Simulation Layer

Purpose

This layer converts:

- operational cost intelligence
  into:
- live project pricing recommendations.

This becomes:

the execution layer of the pricing engine.

The simulation system should feel:

- reactive
- intelligent
- financially aware
- operationally grounded

NOT:

- calculator-like
- spreadsheet-like
- static form based.

---

37. Smart Estimate Workflow

MASTER FLOW

Project Inputs

↓

Resource Mapping

↓

Operational Cost Computation

↓

Contingency Application

↓

Margin Simulation

↓

Billing Rate Generation

↓

Benchmark Validation

↓

Risk Analysis

↓

Pricing Recommendation

---

38. Project Entity Architecture

ENTITY: ProjectEstimate

Fields

Field	Type
id	UUID
project_name	String
client_name	String
client_region	String
project_type	Enum
estimated_hours	Number
delivery_timeline_days	Number
contract_type	Enum
target_margin	Number
contingency_percent	Number
recommended_quote	Number
billing_rate	Number
risk_score	Number
profitability_score	Number
status	Enum
created_by	UUID
created_at	Timestamp

---

39. Resource Allocation System

Purpose

Projects consume:

- multiple employees
- infra
- AI tooling
- operational bandwidth

The system must support:

mixed allocation modeling.

ENTITY: ProjectResourceAllocation

Fields

Field	Type
id	UUID
project_id	UUID
employee_role_id	UUID
quantity	Number
allocated_hours	Number
utilization_factor	Number
calculated_cost	Number

Resource Allocation Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Resource\ Cost = Effective\ Hourly\ Cost \times Allocated\ Hours \times Quantity"}}

---

40. Total Operational Cost Computation

Purpose

Aggregate ALL delivery-related operational expenses.

This becomes:

the true project delivery cost.

Cost Components

Workforce

- developers
- QA
- PM
- design
- DevOps

Infrastructure

- hosting
- APIs
- cloud services

AI & SaaS

- AI tools
- subscriptions
- delivery tooling

Overheads

- admin
- HR
- compliance
- operations

Master Operational Cost Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Total\ Operational\ Cost = Workforce + Infrastructure + SaaS + Overheads"}}

---

41. Delivery Timeline Intelligence

Purpose

Delivery duration directly impacts:

- operational burden
- staffing overlap
- infrastructure dependency
- payment recovery
- delivery pressure

Timeline Risk Logic

Short timelines increase:

- delivery pressure
- coordination overhead

- burnout probability
- pricing volatility

Long timelines increase:

- operational drift
- management overhead
- delayed profitability

Timeline Intelligence Outputs

Outputs

- healthy timeline
- compressed delivery
- extended recovery risk
- staffing instability

---

42. Contract Type Intelligence

Purpose

Different commercial models require different pricing logic.

Supported Contract Types

Fixed Cost

Highest pricing risk.

Requires:

- strong contingency
- stronger margin protection

Time & Material

Lower delivery risk.

Requires:

- hourly optimization
- utilization management

Retainer

Focus on:

- delivery sustainability
- staffing continuity

Dedicated Team

Focus on:

- utilization efficiency
- long-term margin stability

Contract Risk Multiplier

Each contract type modifies:

- contingency recommendation
- pricing aggressiveness
- risk scoring

---

43. Dynamic Margin Simulation

Purpose

The platform must visually simulate:

- profitability impact
- quote sensitivity
- pricing sustainability

in real time.

Dynamic Simulation Variables

Inputs

- margin changes
- delivery changes
- staffing changes
- infra changes
- milestone changes

Simulation Output Updates

Outputs

- recommended quote
- profit amount
- margin health
- quote competitiveness
- risk level

Simulation Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Profit = Revenue - Operational\ Cost"}}

Margin Simulation Formula

genui {"math_block_widget_always_prefetch_v2":{"content":"Profitability\ Percentage = \frac{Profit}{Revenue} \times 100"}}

---

44. Intelligent Pricing Recommendation Layer

Purpose

The system should recommend:

- optimal pricing
- safe pricing
- aggressive pricing
- enterprise pricing

based on:

- operational data
- benchmark alignment
- risk conditions

Recommendation Types

Safe Recommendation

Operationally sustainable.

Competitive Recommendation

Balanced pricing.

Enterprise Recommendation

Premium positioning.

Aggressive Recommendation

Higher conversion probability.
Lower safety.

Recommendation Inputs

Inputs

- operational burn
- margin target
- client region
- contract type
- delivery complexity
- project duration
- benchmark range

---

45. AI Recommendation Layer

IMPORTANT

AI should NOT generate final calculations.

AI should ONLY:

- explain
- interpret
- recommend
- summarize
- identify patterns

AI Insight Examples

Current pricing remains 12% below US enterprise benchmark.

Delivery timeline introduces elevated operational pressure.

Contingency allocation may be insufficient for AI integration complexity.

AI System Responsibilities

AI SHOULD HANDLE

- pricing explanations
- recommendation narratives
- audit summaries
- benchmark interpretation
- risk commentary

AI SHOULD NOT HANDLE

- deterministic calculations
- margin computation
- operational burn logic
- NPV formulas
- pricing formulas

---

46. Proposal Safety Engine

Purpose

This system validates whether pricing is:

- operationally safe
- commercially realistic
- margin sustainable

Safety Inputs

Inputs

- operational burn
- quote value

- benchmark alignment
- contingency level
- milestone structure
- utilization pressure

Safety Outputs

Results

- Financially Healthy
- Moderate Risk
- Underquoted
- Aggressive Pricing
- Unsustainable

---

47. Quote Confidence Score

Purpose

Provide executive-level pricing confidence.

Confidence Factors

Inputs

- margin health
- benchmark alignment
- contingency quality
- payment recovery
- operational stability
- delivery pressure

Confidence Formula Logic

Weighted scoring model.

Example:

Margin Health = 30%

Operational Stability = 25%

Cash Recovery = 20%

Benchmark Alignment = 25%

Confidence Output

Score

0–100

Confidence Categories

Strong Confidence

80–100

Moderate Confidence

60–79

Weak Confidence

40–59

Critical Risk

Below 40

---

48. Live UI Reactivity Architecture

Purpose

The interface must feel:

alive.

Every pricing modification should visually update:

- instantly
- smoothly
- contextually

Reactive Update Areas

UI Components

- pricing cards
- margin indicators
- graphs

- risk meters
- confidence score
- billing rate
- profitability summary

UI Animation Principles

Use:

- soft number transitions
- smooth graph interpolation
- contextual highlight updates
- subtle glow indicators

Avoid:

- aggressive animation
- gaming aesthetics
- flashy transitions

---

49. Pricing Engine API Architecture

Core APIs

API 1 — Create Estimate

Endpoint

POST /estimates/create

Purpose

Generate estimate simulation.

API 2 — Recalculate Simulation

Endpoint

POST /simulation/recalculate

Purpose

Run live pricing recomputation.

API 3 — Save Operational Config

Endpoint

POST /config/save

API 4 — Load Saved Config

Endpoint

GET /config/{id}

API 5 — Benchmark Retrieval

Endpoint

GET /benchmarks/{region}

API 6 — AI Pricing Insights

Endpoint

POST /ai/pricing-insights

---

50. Recommended Backend Stack

Core Backend

- Antigravity
- PostgreSQL
- Supabase

AI Layer

Recommended:

- OpenRouter
- Gemini API
- Claude via OpenRouter

Visualization

- Recharts
- lightweight graph rendering

State Management

Recommended:

- Zustand
  or
- Context + Reducers

---

51. Database Architecture Recommendation

IMPORTANT

Keep:

modular schema structure.

Avoid:

- giant combined tables
- duplicated financial data
- hardcoded pricing rules

Recommended Tables

organizations

employees

infrastructure_services

saas_tools

overhead_categories

pricing_policies

regional_benchmarks

project_estimates

project_resources

milestone_structures

pricing_simulations

audit_logs

---

52. Audit Logging System

Purpose

Track:

- pricing changes
- operational changes
- quote adjustments
- risk changes

Audit Examples

Target margin changed from 28% → 32%

Infrastructure allocation updated.

Safe billing rate recalculated.

---

53. Pricing Intelligence Dashboard Logic

Purpose

Provide:

- executive pricing visibility
- operational awareness
- quote health overview

Dashboard Core Metrics

Metrics

- Monthly Organizational Burn
- Effective Hourly Burn
- Safe Billing Rate
- Average Margin
- Operational Stability
- Pricing Confidence

---

54. System Scalability Strategy

IMPORTANT

The architecture should support:

- multi-organization support
- multi-region pricing
- future AI learning systems
- historical profitability intelligence
- enterprise scaling

Therefore:
all pricing engines must remain:

modular and isolated.

---

55. Final Product Engineering Principle

The system should NEVER optimize for:

- visual complexity
- fake AI appearance
- flashy dashboards

The system should optimize for:

financial correctness.

Everything else is secondary.

FINAL PRODUCT OUTCOME

At the end of Phase 1, the platform should successfully function as:

a live operational pricing intelligence engine

capable of:

- modeling real organizational burn
- calculating financially safe billing rates
- generating sustainable quotations
- preventing hidden delivery losses
- improving pricing confidence
- simulating profitability in real time

This becomes:

the core foundation for the entire product ecosystem.

END OF PART 3